LAPORAN PRAKTIKUM

STRUKTUR DATA

DOUBLE LINKED LIST

Disusun Oleh:

Gina Ramadhani

Nim: 2511533014

Dosen Pengampu: DR. Wahyudi, S.T, M.T

Asisten Praktikum: Sherly Sukmadira

DAPERTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum yang berjudul *"Double Linked List"* ini dapat diselesaikan dengan baik. Laporan ini dibuat sebagai bentuk dokumentasi dan pemahaman penulis terhadap materi struktur data khususnya Double Linked List dalam bahasa pemrograman Java.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar laporan ini menjadi lebih baik. Semoga laporan ini dapat bermanfaat bagi pembaca dan menambah wawasan mengenai struktur data Double Linked List.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR ..... **i**
DAFTAR ISI ..... **ii**
BAB I PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat Praktikum 1
BAB II PEMBAHASAN **2**
2.1 Dasar Teori 2
2.2 Praktikum Class “NodeDLL_2511533014” 2
2.3 Praktikum Class “InsertDLL_2511533014” 4
2.4 Praktikum Class “PenelusuranDLL_2511533014” 12
2.5 Praktikum Class “HapusDLL_2511533014” 17
2.6 Laporan Tugas Praktikum 25
BAB III PENUTUP **39**
3.1 Kesimpulan 39
DAFTAR PUSTAKA **40**

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam pemrograman karena digunakan untuk mengatur dan menyimpan data secara efisien. Salah satu struktur data yang sering digunakan adalah Linked List. Double Linked List merupakan pengembangan dari Single Linked List yang memiliki dua pointer, yaitu pointer menuju node berikutnya dan node sebelumnya. Dengan adanya dua pointer tersebut, proses traversal data dapat dilakukan dari depan maupun belakang. Pada praktikum ini dilakukan implementasi Double Linked List menggunakan bahasa pemrograman Java untuk memahami cara kerja penambahan, penghapusan, pencarian, dan penampilan data pada struktur data tersebut.

1.2 Tujuan Praktikum

1. Memahami konsep dasar Double Linked List.
2. Mengimplementasikan Double Linked List menggunakan bahasa Java.
3. Mengetahui proses penambahan dan penghapusan node pada Double Linked List.
4. Memahami traversal maju dan mundur pada Double Linked List.

1.3 Manfaat Praktikum

1. Menambah pemahaman tentang struktur data Double Linked List.
2. Melatih kemampuan pemrograman Java dalam pengolahan data dinamis.
3. Memahami penggunaan pointer next dan prev.
4. Membantu memahami konsep traversal data dua arah.

BAB II
PEMBAHASAN

2.1 Dasar Teori

Double Linked List adalah struktur data linear yang terdiri dari kumpulan node dimana setiap node memiliki data, pointer ke node berikutnya (next), dan pointer ke node sebelumnya (prev). Struktur ini memungkinkan proses traversal dilakukan dari depan ke belakang maupun sebaliknya. Dibandingkan Single Linked List, Double Linked List lebih fleksibel dalam manipulasi data karena dapat bergerak dua arah. Operasi dasar pada Double Linked List meliputi penambahan node, penghapusan node, pencarian data, dan penampilan data secara maju maupun mundur. Dalam implementasinya menggunakan Java, setiap node direpresentasikan sebagai object yang saling terhubung menggunakan reference.

2.2 Praktikum Class "NodeDLL_2511533014"

```java
package pekan6_2511533014;

public class NodeDLL_2511533014 {
    // mendefinisikan kelas Node
    int data_3014;  // data
    NodeDLL_2511533014 next_3014;  // Pointer ke next node
    NodeDLL_2511533014 prev_3014;  // Pointer ke previous node

    // konstruktor
    public NodeDLL_2511533014 (int data_3014)  {
        this.data_3014 = data_3014;
        this.next_3014 = null;
        this.prev_3014 = null;
    }
}
```

Gambar 2.1 Kode Program Praktikum Class "NodeDLL"

Program di atas merupakan kelas NodeDLL_2511533014 yang digunakan untuk membuat node pada struktur data Double Linked List (DLL). Setiap node memiliki data bertipe integer serta dua pointer, yaitu next_3014 untuk menunjuk node berikutnya dan prev_3014 untuk menunjuk node sebelumnya. Konstruktor digunakan untuk mengisi nilai data saat objek node dibuat serta mengatur pointer awal menjadi null.

Berikut penjelasan langkah-langkah kerjanya:

1. package pekan6_2511533014;

   → Digunakan untuk menentukan package atau folder tempat kelas disimpan.
2. public class NodeDLL_2511533014 {

   → Mendeklarasikan kelas bernama NodeDLL_2511533014 yang digunakan sebagai node pada Doubly Linked List.
3. // mendefinisikan kelas Node

   → Komentar untuk menjelaskan bahwa bagian ini digunakan untuk membuat node.
4. int data_3014;

   → Variabel untuk menyimpan data bertipe integer pada node.
5. NodeDLL_2511533014 next_3014;

   → Variabel pointer yang digunakan untuk menunjuk node berikutnya.
6. NodeDLL_2511533014 prev_3014;

   → Variabel pointer yang digunakan untuk menunjuk node sebelumnya.
7. // konstruktor

   → Komentar yang menjelaskan bahwa bagian berikut adalah konstruktor kelas.
8. public NodeDLL_2511533014 (int data_3014) {

   → Konstruktor kelas yang dipanggil saat objek node dibuat dan menerima parameter data.
9. this.data_3014 = data_3014;

   → Mengisi variabel data_3014 pada objek dengan nilai parameter yang diberikan.
10. this.next_3014 = null;

    → Mengatur pointer next_3014 menjadi null karena node belum terhubung ke node lain.
11. this.prev_3014 = null;

    → Mengatur pointer prev_3014 menjadi null karena node belum memiliki node sebelumnya.
12. }

    → Penutup konstruktor.

13. }

→ Penutup kelas NodeDLL_2511533014.

2.3 Praktikum Class "InsertDLL_2511533014"

```java
package pekan6_2511533014;

public class InsertDLL_2511533014 {
    // menambahkan node di awal DLL
    static NodeDLL_2511533014 insertBegin_3014(NodeDLL_2511533014 head_3014, int data_3014){
        NodeDLL_2511533014 new_node_3014 = new NodeDLL_2511533014(data_3014);
        new_node_3014.next_3014 = head_3014;
        if (head_3014 != null){
            head_3014.prev_3014 = new_node_3014;
        }
        return new_node_3014;
    }

    // fungsi menambahkan node di akhir
    public static NodeDLL_2511533014 insertEnd_3014(NodeDLL_2511533014 head_3014, int newData_3014){
        NodeDLL_2511533014 newNode_3014 = new NodeDLL_2511533014(newData_3014);
        if (head_3014 == null){
            head_3014 = newNode_3014;
        } else {
            NodeDLL_2511533014 curr_3014 = head_3014;
            while (curr_3014.next_3014 != null){
                curr_3014 = curr_3014.next_3014;
            }
            curr_3014.next_3014 = newNode_3014;
            newNode_3014.prev_3014 = curr_3014;
        }
        return head_3014;
    }
```

```java
    // fungsi menambahkan node di posisi tertentu
    public static NodeDLL_2511533014 insertAtPosition_3014(NodeDLL_2511533014 head_3014, int pos_3014, int new_data_3014) {
        NodeDLL_2511533014 new_node_3014 = new NodeDLL_2511533014(new_data_3014);
        if (pos_3014 == 1) {
            new_node_3014.next_3014 = head_3014;
            if (head_3014 != null) {
                head_3014.prev_3014 = new_node_3014;
            }
            head_3014 = new_node_3014;
            return head_3014;
        }
        NodeDLL_2511533014 curr_3014 = head_3014;
        for (int i_3014 = 1; i_3014 < pos_3014 - 1 && curr_3014 != null; ++i_3014) {
            curr_3014 = curr_3014.next_3014;
        }
        if (curr_3014 == null) {
            System.out.println("Posisi tidak ada");
            return head_3014;
        }
        new_node_3014.prev_3014 = curr_3014;
        new_node_3014.next_3014 = curr_3014.next_3014;
        curr_3014.next_3014 = new_node_3014;
        if (new_node_3014.next_3014 != null) {
            new_node_3014.next_3014.prev_3014 = new_node_3014;
        }
        return head_3014;
    }
```

```java
    public static void printList_3014(NodeDLL_2511533014 head_3014) {
        NodeDLL_2511533014 curr_3014 = head_3014;
        while (curr_3014 != null) {
            System.out.print(curr_3014.data_3014 + " <-> ");
            curr_3014 = curr_3014.next_3014;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        NodeDLL_2511533014 head_3014 = new NodeDLL_2511533014(2);
        head_3014.next_3014 = new NodeDLL_2511533014(3);
        head_3014.next_3014.prev_3014 = head_3014;
        head_3014.next_3014.next_3014 = new NodeDLL_2511533014(5);
        head_3014.next_3014.next_3014.prev_3014 = head_3014.next_3014;

        System.out.print("DLL Awal: ");
        printList_3014(head_3014);

        head_3014 = insertBegin_3014(head_3014, 1);
        System.out.print("Simpul 1 ditambah di awal: ");
        printList_3014(head_3014);

        System.out.print("Simpul 6 ditambah di akhir: ");
        int data_3014 = 6;
        head_3014 = insertEnd_3014(head_3014, data_3014);
        printList_3014(head_3014);

        System.out.print("Tambah node 4 di posisi 4: ");
        int data2_3014 = 4;
        int pos_3014 = 4;
        head_3014 = insertAtPosition_3014(head_3014, pos_3014, data2_3014);
        printList_3014(head_3014);
    }
}
```

Gambar 2.2 Kode Program Praktikum Class "InsertDLL"

Program InsertDLL_2511533014 digunakan untuk mengimplementasikan operasi penambahan node pada struktur data Double Linked List (DLL). Program ini memiliki fungsi untuk menambah node di awal, di akhir, dan pada posisi tertentu dalam linked list. Selain itu, terdapat fungsi untuk menampilkan isi linked list serta method main untuk menjalankan dan menguji semua operasi penambahan node tersebut.

Berikut penjelasan langkah-langkah kerjanya:

1. package pekan6_2511533014

   → Menentukan package tempat program disimpan.

2. public class InsertDLL_2511533014 {

   → Mendeklarasikan kelas InsertDLL_2511533014.

3. // menambahkan node di awal DLL

   → Komentar penjelasan fungsi untuk menambah node di awal.

4. static NodeDLL_2511533014 insertBegin_3014(NodeDLL_2511533014 head_3014, int data_3014){
   → Mendeklarasikan method untuk menambahkan node di awal DLL.
5. NodeDLL_2511533014 new_node_3014 = new NodeDLL_2511533014(data_3014);
   → Membuat node baru dengan data yang diberikan.
6. new_node_3014.next_3014 = head_3014;
   → Menghubungkan node baru ke head lama.
7. if (head_3014 != null){
   → Mengecek apakah linked list tidak kosong.
8. head_3014.prev_3014 = new_node_3014;
   → Menghubungkan pointer previous head lama ke node baru.
9. }
   → Penutup percabangan if.
10. return new_node_3014;
    → Mengembalikan node baru sebagai head baru.
11. }
    → Penutup method insertBegin_3014.
12. // fungsi menambahkan node di akhir
    → Komentar penjelasan fungsi tambah node di akhir.
13. public static NodeDLL_2511533014 insertEnd_3014(NodeDLL_2511533014 head_3014, int newData_3014){
    → Mendeklarasikan method tambah node di akhir DLL.
14. NodeDLL_2511533014 newNode_3014 = new NodeDLL_2511533014(newData_3014);
    → Membuat node baru.
15. if (head_3014 == null){
    → Mengecek apakah linked list kosong.
16. head_3014 = newNode_3014;
    → Jika kosong, node baru menjadi head.
17. } else {
    → Jika tidak kosong maka masuk ke proses berikutnya.

18. NodeDLL_2511533014 curr_3014 = head_3014;
    → Membuat variabel bantu untuk traversal.
19. while (curr_3014.next_3014 != null){
    → Perulangan untuk mencari node terakhir.
20. curr_3014 = curr_3014.next_3014;
    → Pindah ke node berikutnya.
21. }
    → Penutup perulangan while.
22. curr_3014.next_3014 = newNode_3014;
    → Menghubungkan node terakhir ke node baru.
23. newNode_3014.prev_3014 = curr_3014;
    → Menghubungkan pointer previous node baru ke node terakhir.
24. }
    → Penutup blok else.
25. return head_3014;
    → Mengembalikan head linked list.
26. }
    → Penutup method insertEnd_3014.
27. // fungsi menambahkan node di posisi tertentu
    → Komentar penjelasan method tambah node di posisi tertentu.
28. public static NodeDLL_2511533014 insertAtPosition_3014(NodeDLL_2511533014 head_3014, int pos_3014, int new_data_3014) {
    → Mendeklarasikan method tambah node berdasarkan posisi.
29. NodeDLL_2511533014 new_node_3014 = new NodeDLL_2511533014(new_data_3014);
    → Membuat node baru.
30. if (pos_3014 == 1) {
    → Mengecek apakah posisi adalah awal linked list.
31. new_node_3014.next_3014 = head_3014;
    → Menghubungkan node baru ke head lama.
32. if (head_3014 != null) {

→ Mengecek apakah linked list tidak kosong.

33. head_3014.prev_3014 = new_node_3014;

→ Menghubungkan previous head lama ke node baru.

34. }

→ Penutup if.

35. head_3014 = new_node_3014;

→ Menjadikan node baru sebagai head.

36. return head_3014;

→ Mengembalikan head baru.

37. }

→ Penutup if posisi 1.

38. NodeDLL_2511533014 curr_3014 = head_3014;

→ Variabel bantu traversal.

39. for (int i_3014 = 1; i_3014 < pos_3014 - 1 && curr_3014 != null; ++i_3014) {

→ Perulangan untuk menuju posisi sebelum node yang akan disisipkan.

40. curr_3014 = curr_3014.next_3014;

→ Pindah ke node berikutnya.

41. }

→ Penutup perulangan for.

42. if (curr_3014 == null) {

→ Mengecek apakah posisi tidak tersedia.

43. System.out.println("Posisi tidak ada");

→ Menampilkan pesan jika posisi tidak ditemukan.

44. return head_3014;

→ Mengembalikan head tanpa perubahan.

45. }

→ Penutup if.

46. new_node_3014.prev_3014 = curr_3014;

→ Menghubungkan previous node baru ke node sebelumnya.

47. new_node_3014.next_3014 = curr_3014.next_3014;

→ Menghubungkan next node baru ke node sesudahnya.

48. curr_3014.next_3014 = new_node_3014;
    → Menghubungkan node sebelumnya ke node baru.
49. if (new_node_3014.next_3014 != null) {
    → Mengecek apakah node baru bukan node terakhir.
50. new_node_3014.next_3014.prev_3014 = new_node_3014;
    → Menghubungkan previous node setelahnya ke node baru.
51. }
    → Penutup if.
52. return head_3014;
    → Mengembalikan head linked list.
53. }
    → Penutup method insertAtPosition_3014.
54. public static void printList_3014(NodeDLL_2511533014 head_3014) {
    → Method untuk menampilkan isi DLL.
55. NodeDLL_2511533014 curr_3014 = head_3014;
    → Variabel bantu traversal.
56. while (curr_3014 != null) {
    → Perulangan selama node masih ada.
57. System.out.print(curr_3014.data_3014 + " <-> ");
    → Menampilkan data node.
58. curr_3014 = curr_3014.next_3014;
    → Pindah ke node berikutnya.
59. }
    → Penutup while.
60. System.out.println();
    → Pindah ke baris baru.
61. }
    → Penutup method printList_3014.
62. public static void main(String[] args) {
    → Method utama program.
63. NodeDLL_2511533014 head_3014 = new NodeDLL_2511533014(2);
    → Membuat node pertama dengan data 2.

64. head_3014.next_3014 = new NodeDLL_2511533014(3);
    → Menambahkan node kedua dengan data 3.
65. head_3014.next_3014.prev_3014 = head_3014;
    → Menghubungkan previous node kedua ke node pertama.
66. head_3014.next_3014.next_3014 = new NodeDLL_2511533014(5);
    → Menambahkan node ketiga dengan data 5.
67. head_3014.next_3014.next_3014.prev_3014 = head_3014.next_3014;
    → Menghubungkan previous node ketiga ke node kedua.
68. System.out.print("DLL Awal: ");
    → Menampilkan tulisan “DLL Awal”.
69. printList_3014(head_3014);
    → Menampilkan isi DLL awal.
70. head_3014 = insertBegin_3014(head_3014, 1);
    → Menambahkan node bernilai 1 di awal DLL.
71. System.out.print("Simpul 1 ditambah di awal: ");
    → Menampilkan keterangan penambahan node awal.
72. printList_3014(head_3014);
    → Menampilkan DLL setelah penambahan awal.
73. System.out.print("Simpul 6 ditambah di akhir: ");
    → Menampilkan keterangan penambahan node akhir.
74. int data_3014 = 6;
    → Menyimpan nilai 6 ke variabel.
75. head_3014 = insertEnd_3014(head_3014, data_3014);
    → Menambahkan node 6 di akhir DLL.
76. printList_3014(head_3014);
    → Menampilkan DLL setelah penambahan akhir.
77. System.out.print("Tambah node 4 di posisi 4: ");
    → Menampilkan keterangan penambahan node pada posisi tertentu.
78. int data2_3014 = 4;
    → Menyimpan nilai 4.
79. int pos_3014 = 4;
    → Menyimpan posisi ke-4.

80. head_3014 = insertAtPosition_3014(head_3014, pos_3014, data2_3014);
    → Menambahkan node 4 pada posisi ke-4.
81. printList_3014(head_3014);
    → Menampilkan DLL setelah penambahan posisi tertentu.
82. }
    → Penutup method main.
83. }
    → Penutup kelas InsertDLL_2511533014.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.3 dibawah ini.

```
<terminated> InsertDLL_2511533014 [Java Application] C:\Users\rara0\.p2\pool\plugins\o
DLL Awal: 2 <-> 3 <-> 5 <->
Simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
Simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
Tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

Gambar 2.3 Output kode program praktikum InsertDLL

2.3.1 Analisis

Program menghasilkan output berupa perubahan isi Doubly Linked List (DLL) setelah dilakukan beberapa operasi penambahan node. Awalnya DLL berisi data 2, 3, 5, kemudian ditambahkan node 1 di awal, node 6 di akhir, dan node 4 pada posisi ke-4. Hasil akhir menunjukkan urutan data menjadi 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6, yang menandakan bahwa proses insert di awal, akhir, dan posisi tertentu berhasil dilakukan dengan benar serta hubungan antar node pada DLL tetap terhubung dua arah menggunakan pointer next dan prev.

Output menunjukkan bahwa seluruh operasi pada Doubly Linked List berjalan dengan benar, yaitu:

- penambahan node di awal,
- penambahan node di akhir,
- dan penyisipan node pada posisi tertentu.

Setiap perubahan berhasil memperbarui hubungan antar node menggunakan pointer next dan prev.

2.4 Praktikum Class "PenelusuranDLL_2511533014"

```java
package pekan6_2511533014;

public class PenelusuranDLL_2511533014 {
    //fungsi penelurusan maju
    static void forwardTraversal_3014 (NodeDLL_2511533014 head_3014) {
        //memulai penelusuran dari head
        NodeDLL_2511533014 curr_3014 = head_3014;
        //lanjutkan sampai akhir
        while (curr_3014 != null) {
            //print data
            System.out.print (curr_3014.data_3014 + " <-> ");
            //pindahkan ke node berikutnya
            curr_3014 = curr_3014.next_3014;
        }
        //print spasi
        System.out.println ();
    }
    //fungsi penelusuran mundur
    static void backwardTraversal_3014 (NodeDLL_2511533014 tail_3014) {
        //mulai dari akhir
        NodeDLL_2511533014 curr_3014 = tail_3014;
        //lanjut sampai head
        while (curr_3014 != null) {
            //cetak data
            System.out.print (curr_3014.data_3014 + " <-> ");
            //pindahkan ke node sebelumnya
            curr_3014 = curr_3014.prev_3014;
        }
        //cetak spasi
        System.out.println();
    }
```

```java
    public static void main (String[] args) {
        //cetak dll
        NodeDLL_2511533014 head_3014 = new NodeDLL_2511533014 (1);
        NodeDLL_2511533014 second_3014 = new NodeDLL_2511533014 (2);
        NodeDLL_2511533014 third_3014 = new NodeDLL_2511533014 (3);

        head_3014.next_3014 = second_3014;
        second_3014.prev_3014 = head_3014;
        second_3014.next_3014 = third_3014;
        third_3014.prev_3014 = second_3014;

        System.out.println ("Penelusuran maju: ");
        forwardTraversal_3014 (head_3014);

        System.out.println ("Penelusuran mundur: ");
        backwardTraversal_3014 (third_3014);

    }
}
```

Gambar 2.4 Kode Program Praktikum Class "PenelusuranDLL"

Program PenelusuranDLL_2511533014 digunakan untuk melakukan penelusuran atau traversal pada struktur data Doubly Linked List (DLL). Program ini memiliki dua method, yaitu forwardTraversal_3014() untuk menelusuri node dari awal ke akhir menggunakan pointer next, dan backwardTraversal_3014() untuk menelusuri node dari akhir ke awal menggunakan pointer prev. Pada method main, dibuat tiga node yang saling terhubung, kemudian dilakukan penelusuran maju dan mundur untuk menampilkan isi linked list.

Berikut penjelasan langkah-langkah kerjanya:

1. package pekan6_2511533014;
   → Menentukan package tempat program disimpan.
2. public class PenelusuranDLL_2511533014 {
   → Mendeklarasikan kelas PenelusuranDLL_2511533014.
3. //fungsi penelurusan maju
   → Komentar penjelasan fungsi traversal maju.
4. static void forwardTraversal_3014 (NodeDLL_2511533014 head_3014) {
   → Mendeklarasikan method untuk traversal maju pada DLL.
5. //memulai penelusuran dari head
   → Komentar bahwa traversal dimulai dari head.
6. NodeDLL_2511533014 curr_3014 = head_3014;
   → Membuat variabel bantu curr_3014 yang menunjuk ke head.
7. //lanjutkan sampai akhir
   → Komentar bahwa traversal dilakukan sampai node terakhir.
8. while (curr_3014 != null) {
   → Perulangan selama node masih ada.
9. //print data
   → Komentar untuk mencetak data node.
10. System.out.print (curr_3014.data_3014 + " <-> ");
    → Menampilkan data node beserta tanda penghubung.
11. //pindahkan ke node berikutnya

→ Komentar perpindahan ke node berikutnya.

12. curr_3014 = curr_3014.next_3014;

→ Memindahkan pointer ke node selanjutnya.

13. }

→ Penutup perulangan while.

14. //print spasi

→ Komentar untuk mencetak baris baru.

15. System.out.println ();

→ Mencetak pindah baris setelah traversal selesai.

16. }

→ Penutup method forwardTraversal_3014.

17. //fungsi penelusuran mundur

→ Komentar penjelasan traversal mundur.

18. static void backwardTraversal_3014 (NodeDLL_2511533014 tail_3014) {

→ Mendeklarasikan method traversal mundur.

19. //mulai dari akhir

→ Komentar bahwa traversal dimulai dari tail.

20. NodeDLL_2511533014 curr_3014 = tail_3014;

→ Variabel bantu menunjuk ke node terakhir.

21. //lanjut sampai head

→ Komentar traversal dilakukan sampai awal linked list.

22. while (curr_3014 != null) {

→ Perulangan selama node masih ada.

23. //cetak data

→ Komentar untuk mencetak data node.

24. System.out.print (curr_3014.data_3014 + " <-> ");

→ Menampilkan data node.

25. //pindahkan ke node sebelumnya

→ Komentar perpindahan ke node sebelumnya.

26. curr_3014 = curr_3014.prev_3014;

→ Memindahkan pointer ke node sebelumnya.

27. }

→ Penutup perulangan while.

28. //cetak spasi

→ Komentar untuk pindah baris.

29. System.out.println();

→ Mencetak baris baru setelah traversal selesai.

30. }

→ Penutup method backwardTraversal_3014.

31. public static void main (String[] args) {

→ Method utama program.

32. //cetak dll

→ Komentar untuk membuat DLL.

33. NodeDLL_2511533014 head_3014 = new NodeDLL_2511533014 (1);

→ Membuat node pertama dengan data 1.

34. NodeDLL_2511533014 second_3014 = new NodeDLL_2511533014 (2);

→ Membuat node kedua dengan data 2.

35. NodeDLL_2511533014 third_3014 = new NodeDLL_2511533014 (3);

→ Membuat node ketiga dengan data 3.

36. head_3014.next_3014 = second_3014;

→ Menghubungkan node pertama ke node kedua.

37. second_3014.prev_3014 = head_3014;

→ Menghubungkan previous node kedua ke node pertama.

38. second_3014.next_3014 = third_3014;

→ Menghubungkan node kedua ke node ketiga.

39. third_3014.prev_3014 = second_3014;

→ Menghubungkan previous node ketiga ke node kedua.

40. System.out.println ("Penelusuran maju: ");

→ Menampilkan teks “Penelusuran maju”.

41. forwardTraversal_3014 (head_3014);

→ Memanggil method traversal maju.

42. System.out.println ("Penelusuran mundur: ");

→ Menampilkan teks “Penelusuran mundur”.

43. backwardTraversal_3014 (third_3014);

→ Memanggil method traversal mundur dimulai dari node terakhir.

44. }

→ Penutup method main.

45. }

→ Penutup kelas PenelusuranDLL_2511533014.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.5 dibawah ini.

```
<terminated> PenelusuranDLL_2511533014 [
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

Gambar 2.5 Output kode program praktikum

class “PenelusuranDLL”

2.3.2 Analisis

Program menghasilkan output berupa proses penelusuran pada struktur data Doubly Linked List (DLL) secara maju dan mundur. Pada penelusuran maju, data ditampilkan mulai dari node pertama hingga node terakhir menggunakan pointer next, sehingga menghasilkan urutan 1 <-> 2 <-> 3 <->. Sedangkan pada penelusuran mundur, data ditampilkan dari node terakhir ke node pertama menggunakan pointer prev, sehingga menghasilkan urutan 3 <-> 2 <-> 1 <->. Output ini menunjukkan bahwa hubungan antar node pada DLL telah terhubung dengan benar dalam dua arah, sehingga traversal dapat dilakukan maju maupun mundur.

2.5 Praktikum Class "HapusDLL_2511533014"

```java
package pekan6_2511533014;

public class HapusDLL_2511533014 {
    //fungsi mengahpus node di awal
    public static NodeDLL_2511533014 delHead_3014 (NodeDLL_2511533014 head_3014) {
        if (head_3014 == null) {
            return null;
        }
        NodeDLL_2511533014 temp_3014 = head_3014;
        head_3014 = head_3014.next_3014;
        if (head_3014 != null) {
            head_3014.prev_3014 = null;
        }
        return head_3014;
    }
    //fungsi menghapus di akhir
    public static NodeDLL_2511533014 delLast_3014 (NodeDLL_2511533014 head_3014) {
        if (head_3014 == null) {
            return null;
        }
        if (head_3014.next_3014 == null) {
            return null;
        }
        NodeDLL_2511533014 curr_3014 = head_3014;
        while (curr_3014.next_3014 != null) {
            curr_3014 = curr_3014.next_3014;
        }
        //update pointer previous node
        if (curr_3014.prev_3014 != null) {
            curr_3014.prev_3014.next_3014 = null;
        }
        return head_3014;
    }
```

```java
    //fungsi menghapus node posisi tertentu
    public static NodeDLL_2511533014 delPos_3014 (NodeDLL_2511533014 head_3014, int pos_3014) {
        //jika DLL kosong
        if (head_3014 == null) {
            return head_3014;
        }
        NodeDLL_2511533014 curr_3014 = head_3014;
        //telusuri sampai ke node yang akan dihapus
        for (int i = 1; curr_3014 != null && i < pos_3014; ++i) {
            curr_3014 = curr_3014.next_3014;
        }
        //jika posisi tidak ditemukan
        if (curr_3014 == null) {
            return head_3014;
        }
        //update pointer
        if (curr_3014.prev_3014 != null) {
            curr_3014.prev_3014.next_3014 = curr_3014.next_3014;
        }
        if (curr_3014.next_3014 != null) {
            curr_3014.next_3014.prev_3014 = curr_3014.prev_3014;
        }
        //jika yang dihapus head
        if (head_3014 == curr_3014) {
            head_3014 = curr_3014.next_3014;
        }
        return head_3014;
    }
```

```java
    //fungsi mencetak DLL
    public static void printList_3014 (NodeDLL_2511533014 head_3014) {
        NodeDLL_2511533014 curr_3014 = head_3014;
        while (curr_3014 != null) {
            System.out.print ( curr_3014.data_3014 + " <-> ");
            curr_3014 = curr_3014.next_3014;
        }
        System.out.println ();
    }
    public static void main (String[] args) {
        //buat sebuah DLL
        NodeDLL_2511533014 head_3014 = new NodeDLL_2511533014 (1);
        head_3014.next_3014 = new NodeDLL_2511533014 (2);
        head_3014.next_3014.prev_3014 = head_3014;
        head_3014.next_3014.next_3014 = new NodeDLL_2511533014 (3);
        head_3014.next_3014.next_3014.prev_3014 = head_3014.next_3014;
        head_3014.next_3014.next_3014.next_3014 = new NodeDLL_2511533014 (4);
        head_3014.next_3014.next_3014.next_3014.prev_3014 = head_3014.next_3014;
        head_3014.next_3014.next_3014.next_3014.next_3014 = new NodeDLL_2511533014 (5);
        head_3014.next_3014.next_3014.next_3014.next_3014.prev_3014 = head_3014.next_3014.next_3014.next_3014;

        System.out.print ("DLL Awal: ");
        printList_3014(head_3014);

        System.out.print ("Setelah head dihapus: ");
        head_3014 = delHead_3014 (head_3014);
        printList_3014 (head_3014);

        System.out.print("Setelah node terkahir dihapus: ");
        head_3014 = delLast_3014 (head_3014);
        printList_3014 (head_3014);

        System.out.print ("Menghapus node ke 2: ");
        head_3014 = delPos_3014 (head_3014, 2);
        printList_3014 (head_3014);
```

Gambar 2.6 Kode Program Praktikum Class "HapusDLL"

Program HapusDLL_2511533014 digunakan untuk mengimplementasikan operasi penghapusan node pada struktur data Doubly Linked List (DLL). Program ini memiliki fungsi untuk menghapus node di awal (head), di akhir (tail), dan pada posisi tertentu. Selain itu, terdapat method untuk menampilkan isi linked list setelah proses penghapusan dilakukan. Pada method main, program membuat DLL berisi beberapa node lalu melakukan berbagai operasi delete untuk melihat perubahan isi linked list.

Berikut penjelasan langkah-langkah kerjanya:

1. package pekan6_2511533014;

   → Menentukan package tempat program disimpan.

2. public class HapusDLL_2511533014 {

   → Mendeklarasikan kelas HapusDLL_2511533014.

3. //fungsi mengahpus node di awal

   → Komentar penjelasan fungsi hapus node di awal.

4. public static NodeDLL_2511533014 delHead_3014 (NodeDLL_2511533014 head_3014) {

→ Mendeklarasikan method untuk menghapus node head.

5. if (head_3014 == null) {

→ Mengecek apakah DLL kosong.

6. return null;

→ Jika kosong maka mengembalikan null.

7. }

→ Penutup percabangan if.

8. NodeDLL_2511533014 temp_3014 = head_3014;

→ Menyimpan head lama ke variabel sementara.

9. head_3014 = head_3014.next_3014;

→ Memindahkan head ke node berikutnya.

10. if (head_3014 != null) {

→ Mengecek apakah masih ada node setelah head dihapus.

11. head_3014.prev_3014 = null;

→ Menghapus hubungan previous pada head baru.

12. }

→ Penutup if.

13. return head_3014;

→ Mengembalikan head baru.

14. }

→ Penutup method delHead_3014.

15. //fungsi menghapus di akhir

→ Komentar penjelasan fungsi hapus node terakhir.

16. public static NodeDLL_2511533014 delLast_3014 (NodeDLL_2511533014 head_3014) {

→ Mendeklarasikan method hapus node terakhir.

17. if (head_3014 == null) {

→ Mengecek apakah DLL kosong.

18. return null;

→ Jika kosong maka mengembalikan null.

19. }

→ Penutup if.

20. if (head_3014.next_3014 == null) {
    → Mengecek apakah hanya ada satu node.
21. return null;
    → Jika hanya satu node maka DLL menjadi kosong.
22. }
    → Penutup if.
23. NodeDLL_2511533014 curr_3014 = head_3014;
    → Membuat variabel bantu traversal.
24. while (curr_3014.next_3014 != null) {
    → Perulangan untuk menuju node terakhir.
25. curr_3014 = curr_3014.next_3014;
    → Pindah ke node berikutnya.
26. }
    → Penutup while.
27. //update pointer previous node
    → Komentar untuk memperbarui pointer.
28. if (curr_3014.prev_3014 != null) {
    → Mengecek apakah node sebelumnya ada.
29. curr_3014.prev_3014.next_3014 = null;
    → Menghapus hubungan next dari node sebelum tail.
30. }
    → Penutup if.
31. return head_3014;
    → Mengembalikan head DLL.
32. }
    → Penutup method delLast_3014.
33. //fungsi menghapus node posisi tertentu
    → Komentar fungsi hapus node berdasarkan posisi.
34. public static NodeDLL_2511533014 delPos_3014 (NodeDLL_2511533014 head_3014, int pos_3014) {
    → Mendeklarasikan method hapus node pada posisi tertentu.
35. //jika DLL kosong

→ Komentar pengecekan DLL kosong.

36. if (head_3014 == null) {

→ Mengecek apakah DLL kosong.

37. return head_3014;

→ Jika kosong maka mengembalikan head.

38. }

→ Penutup if.

39. NodeDLL_2511533014 curr_3014 = head_3014;

→ Variabel bantu traversal.

40. //telusuri sampai ke node yang akan dihapus

→ Komentar traversal menuju posisi target.

41. for (int i = 1; curr_3014 != null && i < pos_3014; ++i) {

→ Perulangan menuju posisi node yang akan dihapus.

42. curr_3014 = curr_3014.next_3014;

→ Pindah ke node berikutnya.

43. }

→ Penutup for.

44. //jika posisi tidak ditemukan

→ Komentar pengecekan posisi.

45. if (curr_3014 == null) {

→ Mengecek apakah posisi tidak ada.

46. return head_3014;

→ Mengembalikan head tanpa perubahan.

47. }

→ Penutup if.

48. //update pointer

→ Komentar pembaruan pointer.

49. if (curr_3014.prev_3014 != null) {

→ Mengecek apakah node sebelumnya ada.

50. curr_3014.prev_3014.next_3014 = curr_3014.next_3014;

→ Menghubungkan node sebelumnya ke node setelahnya.

51. }

→ Penutup if.

52. if (curr_3014.next_3014 != null) {

→ Mengecek apakah node sesudahnya ada.

53. curr_3014.next_3014.prev_3014 = curr_3014.prev_3014;

→ Menghubungkan previous node sesudahnya ke node sebelumnya.

54. }

→ Penutup if.

55. //jika yang dihapus head

→ Komentar pengecekan jika node yang dihapus adalah head.

56. if (head_3014 == curr_3014) {

→ Mengecek apakah node yang dihapus adalah head.

57. head_3014 = curr_3014.next_3014;

→ Memindahkan head ke node berikutnya.

58. }

→ Penutup if.

59. return head_3014;

→ Mengembalikan head baru.

60. }

→ Penutup method delPos_3014.

61. //fungsi mencetak DLL

→ Komentar fungsi untuk menampilkan DLL.

62. public static void printList_3014 (NodeDLL_2511533014 head_3014) {

→ Method untuk mencetak isi DLL.

63. NodeDLL_2511533014 curr_3014 = head_3014;

→ Variabel bantu traversal.

64. while (curr_3014 != null) {

→ Perulangan selama node masih ada.

65. System.out.print (curr_3014.data_3014 + " <-> ");

→ Menampilkan data node.

66. curr_3014 = curr_3014.next_3014;

→ Pindah ke node berikutnya.

67. }

→ Penutup while.

68. System.out.println ();

→ Pindah ke baris baru.

69. }

→ Penutup method printList_3014.

70. public static void main (String[] args) {

→ Method utama program.

71. //buat sebuah DLL

→ Komentar pembuatan DLL.

72. NodeDLL_2511533014 head_3014 = new NodeDLL_2511533014 (1);

→ Membuat node pertama bernilai 1.

73. head_3014.next_3014 = new NodeDLL_2511533014 (2);

→ Menambahkan node kedua bernilai 2.

74. head_3014.next_3014.prev_3014 = head_3014;

→ Menghubungkan previous node kedua ke node pertama.

75. head_3014.next_3014.next_3014 = new NodeDLL_2511533014 (3);

→ Menambahkan node ketiga bernilai 3.

76. head_3014.next_3014.next_3014.prev_3014 = head_3014.next_3014;

→ Menghubungkan previous node ketiga ke node kedua.

77. head_3014.next_3014.next_3014.next_3014 = new NodeDLL_2511533014 (4);

→ Menambahkan node keempat bernilai 4.

78. head_3014.next_3014.next_3014.next_3014.prev_3014 = head_3014.next_3014;

→ Menghubungkan previous node keempat.

79. head_3014.next_3014.next_3014.next_3014.next_3014 = new NodeDLL_2511533014 (5);

→ Menambahkan node kelima bernilai 5.

80. head_3014.next_3014.next_3014.next_3014.next_3014.prev_3014 = head_3014.next_3014.next_3014.next_3014;

→ Menghubungkan previous node kelima.

81. System.out.print ("DLL Awal: ");
    → Menampilkan tulisan DLL awal.
82. printList_3014(head_3014);
    → Menampilkan isi DLL awal.
83. System.out.print ("Setelah head dihapus: ");
    → Menampilkan keterangan penghapusan head.
84. head_3014 = delHead_3014 (head_3014);
    → Menghapus node pertama.
85. printList_3014 (head_3014);
    → Menampilkan DLL setelah head dihapus.
86. System.out.print("Setelah node terkahir dihapus: ");
    → Menampilkan keterangan penghapusan tail.
87. head_3014 = delLast_3014 (head_3014);
    → Menghapus node terakhir.
88. printList_3014 (head_3014);
    → Menampilkan DLL setelah tail dihapus.
89. System.out.print ("Menghapus node ke 2: ");
    → Menampilkan keterangan penghapusan node posisi tertentu.
90. head_3014 = delPos_3014 (head_3014, 2);
    → Menghapus node pada posisi ke-2.
91. printList_3014 (head_3014);
    → Menampilkan DLL setelah node posisi ke-2 dihapus.
92. }
    → Penutup method main.
93. }
    → Penutup kelas HapusDLL_2511533014.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.7 dibawah ini.

```
<terminated> HapusDLL_2511533014 [Java Application] C:\Users\ran
DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->
Setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->
Setelah node terkahir dihapus: 2 <-> 3 <-> 4 <->
Menghapus node ke 2: 2 <-> 4 <->
```

Gambar 2.7 Output Kode program praktikum class "HapusDLL"

2.3.3 Analisis

Program menghasilkan output yang menunjukkan proses penghapusan node pada struktur data Doubly Linked List (DLL). Awalnya DLL berisi data 1 <-> 2 <-> 3 <-> 4 <-> 5. Kemudian program menghapus node pertama (head) sehingga data 1 hilang dan tersisa 2 <-> 3 <-> 4 <-> 5. Setelah itu node terakhir (tail) yaitu 5 dihapus sehingga DLL menjadi 2 <-> 3 <-> 4. Terakhir, program menghapus node pada posisi ke-2 yaitu data 3, sehingga hasil akhir DLL menjadi 2 <-> 4 <->. Output ini menunjukkan bahwa operasi penghapusan di awal, akhir, dan posisi tertentu berhasil dilakukan dengan benar serta hubungan antar node tetap terhubung menggunakan pointer next dan prev.

2.6 Laporan Tugas Praktikum

2.6.1 Tugas Praktikum Class "Lagu_2511533014"

```java
package pekan6_2511533014;

public class Lagu_2511533014 {

    // Atribut class
     String judul_3014;
     String penyanyi_3014;
    Lagu_2511533014 next_3014;
    Lagu_2511533014 prev_3014;

    // Konstruktor
    public Lagu_2511533014(String judul_3014, String penyanyi_3014) {
        this.judul_3014 = judul_3014;
        this.penyanyi_3014 = penyanyi_3014;
        this.next_3014 = null;
        this.prev_3014 = null;
    }
    // Getter Judul
    public String getJudul_3014() {
        return judul_3014;
    }
    // Setter Judul
    public void setJudul_3014(String judul_3014) {
        this.judul_3014 = judul_3014;
    }
    // Getter Penyanyi
    public String getPenyanyi_3014() {
        return penyanyi_3014;
    }
    // Setter Penyanyi
    public void setPenyanyi_3014(String penyanyi_3014) {
        this.penyanyi_3014 = penyanyi_3014;
    }
}
```

Gambar 2.8 Kode Program Class Lagu

Class Lagu_2511533014 digunakan untuk merepresentasikan satu node pada Double Linked List playlist musik. Class ini menyimpan data lagu berupa judul dan penyanyi, serta memiliki pointer next_3014 untuk menunjuk ke lagu berikutnya dan prev_3014 untuk menunjuk ke lagu sebelumnya. Selain itu, class ini juga memiliki constructor untuk menginisialisasi data lagu dan method getter setter untuk mengambil maupun mengubah data lagu.

Berikut Penjelasan langkah-langkah kerjanya:

1. package pekan6_2511533014;
   → Menentukan package tempat class disimpan.
2. public class Lagu_2511533014 {
   → Mendeklarasikan class bernama Lagu_2511533014.
3. String judul_3014;
   → Variabel untuk menyimpan judul lagu.
4. String penyanyi_3014;
   → Variabel untuk menyimpan nama penyanyi.
5. Lagu_2511533014 next_3014;
   → Pointer untuk menunjuk node atau lagu berikutnya.
6. Lagu_2511533014 prev_3014;
   → Pointer untuk menunjuk node atau lagu sebelumnya.
7. public Lagu_2511533014(String judul_3014, String penyanyi_3014) {
   → Constructor untuk membuat object lagu baru.
8. this.judul_3014 = judul_3014;
   → Mengisi atribut judul dengan nilai input.
9. this.penyanyi_3014 = penyanyi_3014;
   → Mengisi atribut penyanyi dengan nilai input.
10. this.next_3014 = null;
    → Pointer next awalnya kosong/null.
11. this.prev_3014 = null;
    → Pointer prev awalnya kosong/null.

12. }
→ Penutup constructor.
13. public String getJudul_3014() {
→ Method getter untuk mengambil data judul lagu.
14. return judul_3014;
→ Mengembalikan nilai judul lagu.
15. }
→ Penutup method getter judul.
16. public void setJudul_3014(String judul_3014) {
→ Method setter untuk mengubah judul lagu.
17. this.judul_3014 = judul_3014;
→ Mengubah nilai judul lagu.
18. }
→ Penutup method setter judul.
19. public String getPenyanyi_3014() {
→ Method getter untuk mengambil data penyanyi.
20. return penyanyi_3014;
→ Mengembalikan nilai penyanyi lagu.
21. }
→ Penutup method getter penyanyi.
22. public void setPenyanyi_3014(String penyanyi_3014) {
→ Method setter untuk mengubah nama penyanyi.
23. this.penyanyi_3014 = penyanyi_3014;
→ Mengubah nilai penyanyi lagu.
24. }
→ Penutup method setter penyanyi.
25. }
→ Penutup class Lagu_2511533014.

2.6.2 Tugas Praktikum Class "Musik_2511533014"

```java
package pekan6_2511533014;
import java.util.Scanner;
public class Musik_2511533014 {
    static Lagu_2511533014 head_3014 = null;
    static Lagu_2511533014 tail_3014 = null;

    // Method tambah lagu di akhir
    public static void tambahLagu_3014(String judul_3014,String penyanyi_3014) {
        Lagu_2511533014 laguBaru_3014 = new Lagu_2511533014(judul_3014, penyanyi_3014);

        // Jika playlist kosong
        if (head_3014 == null) {
            head_3014 = laguBaru_3014;
            tail_3014 = laguBaru_3014;
        }
        // Jika playlist sudah ada isi
        else {
            tail_3014.next_3014 = laguBaru_3014;
            laguBaru_3014.prev_3014 = tail_3014;
            tail_3014 = laguBaru_3014;
        }
        System.out.println("Lagu berhasil ditambahkan!");
    }
    // Method hapus lagu pertama
    public static void hapusLaguAwal_3014() {
        // Jika playlist kosong
        if (head_3014 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        System.out.println("Lagu "  + head_3014.getJudul_3014() + " berhasil dihapus!");
        // Jika hanya ada satu lagu
        if (head_3014 == tail_3014) {
            head_3014 = null;
            tail_3014 = null;
        }

        // Jika lebih dari satu lagu
        else {
            head_3014 = head_3014.next_3014;
            head_3014.prev_3014 = null;
        }
    }

    // Method tampil playlist maju
    public static void tampilMaju_3014() {
        // Jika playlist kosong
        if (head_3014 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
```

```
        Lagu_2511533014 bantu_3014 = head_3014;
        System.out.println("=== Playlist Maju ===");
        while (bantu_3014 != null) {
            System.out.println("Judul : " + bantu_3014.getJudul_3014());
            System.out.println("Penyanyi : " + bantu_3014.getPenyanyi_3014());
            System.out.println();
            bantu_3014 = bantu_3014.next_3014;
        }
    }
    // Method tampil playlist mundur
    public static void tampilMundur_3014() {
        // Jika playlist kosong
        if (tail_3014 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        Lagu_2511533014 bantu_3014 = tail_3014;
        System.out.println("=== Playlist Mundur ===");
        while (bantu_3014 != null) {
            System.out.println("Judul : "  + bantu_3014.getJudul_3014());
            System.out.println("Penyanyi : " + bantu_3014.getPenyanyi_3014());
            System.out.println();
            bantu_3014 = bantu_3014.prev_3014;
        }
    }
    // Method cari lagu
    public static void cariLagu_3014(String judulCari_3014) {
        // Jika playlist kosong
        if (head_3014 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        Lagu_2511533014 bantu_3014 = head_3014;
        boolean ditemukan_3014 = false;
        while (bantu_3014 != null) {
            if (bantu_3014.getJudul_3014()
                    .equalsIgnoreCase(judulCari_3014)) {
                System.out.println("Lagu ditemukan!");
                System.out.println("Judul : " + bantu_3014.getJudul_3014());
                System.out.println("Penyanyi : " + bantu_3014.getPenyanyi_3014());
                ditemukan_3014 = true;
                break;
            }
            bantu_3014 = bantu_3014.next_3014;
        }
        // Jika lagu tidak ditemukan
        if (ditemukan_3014 == false) {
            System.out.println("Lagu tidak ditemukan!");
        }
    }
    // Main Program
```

```java
    public static void main(String[] args) {
        Scanner input_3014 = new Scanner(System.in);
        int pilihan_3014;
        do {
            System.out.println();
            System.out.println("=== Playlist Musik NIM: 2511533014 ===");
            System.out.println("1. Tambah Lagu");
            System.out.println("2. Hapus Lagu Pertama");
            System.out.println("3. Lihat Playlist Maju");
            System.out.println("4. Lihat Playlist Mundur");
            System.out.println("5. Cari Lagu");
            System.out.println("6. Keluar");
            System.out.print("Pilihan : ");
            pilihan_3014 = input_3014.nextInt();
            input_3014.nextLine();
            // Tambah Lagu
            if (pilihan_3014 == 1) {
                System.out.print("Judul : ");
                String judul_3014 = input_3014.nextLine();
                System.out.print("Penyanyi : ");
                String penyanyi_3014 = input_3014.nextLine();
                tambahLagu_3014(judul_3014, penyanyi_3014);
            }
            // Hapus Lagu
            else if (pilihan_3014 == 2) {
                hapusLaguAwal_3014();
            }
            // Tampil Playlist Maju
            else if (pilihan_3014 == 3) {
                tampilMaju_3014();
            }
            // Tampil Playlist Mundur
            else if (pilihan_3014 == 4) {
                tampilMundur_3014();
            }
            // Cari Lagu
            else if (pilihan_3014 == 5) {
                System.out.print("Masukkan judul lagu : ");
                String cari_3014 = input_3014.nextLine();
                cariLagu_3014(cari_3014);
            }
            // Keluar
            else if (pilihan_3014 == 6) {
                System.out.println("Program selesai!");
            }
            // Jika pilihan salah
            else {
                System.out.println("Pilihan tidak valid!");
            }
        } while (pilihan_3014 != 6);
        input_3014.close();
    }  }
```

Gambar 2.8 Kode Program Class Musik

Class Musik_2511533014 digunakan untuk mengelola playlist musik menggunakan struktur data Double Linked List. Class ini memiliki fitur menambah lagu, menghapus lagu pertama, menampilkan playlist dari depan maupun belakang, mencari lagu berdasarkan judul, serta menu interaktif menggunakan input dari user. Variabel head_3014 menunjuk node pertama dan tail_3014 menunjuk node terakhir pada playlist.

Berikut penjelasan langkah-langkah kerjanya:

1. package pekan6_2511533014;
   → Menentukan package program.
2. import java.util.Scanner;
   → Mengimpor Scanner untuk input user.
3. public class Musik_2511533014 {
   → Mendeklarasikan class Musik_2511533014.
4. static Lagu_2511533014 head_3014 = null;
   → Pointer head untuk menunjuk lagu pertama.
5. static Lagu_2511533014 tail_3014 = null;
   → Pointer tail untuk menunjuk lagu terakhir.
6. public static void tambahLagu_3014(String judul_3014, String penyanyi_3014) {
   → Method untuk menambah lagu baru.
7. Lagu_2511533014 laguBaru_3014 = new Lagu_2511533014(...);
   → Membuat object lagu baru.
8. if (head_3014 == null) {
   → Mengecek apakah playlist kosong.
9. head_3014 = laguBaru_3014;
   → Lagu baru menjadi head.
10. tail_3014 = laguBaru_3014;
    → Lagu baru menjadi tail.
11. else {
    → Jika playlist sudah ada isi.

12. tail_3014.next_3014 = laguBaru_3014;
    → Tail lama menunjuk ke lagu baru.
13. laguBaru_3014.prev_3014 = tail_3014;
    → Lagu baru menunjuk ke tail lama.
14. tail_3014 = laguBaru_3014;
    → Tail dipindahkan ke lagu baru.
15. System.out.println("Lagu berhasil ditambahkan!");
    → Menampilkan pesan berhasil.
16. public static void hapusLaguAwal_3014() {
    → Method untuk menghapus lagu pertama.
17. if (head_3014 == null) {
    → Mengecek apakah playlist kosong.
18. System.out.println("Playlist kosong!");
    → Menampilkan pesan playlist kosong.
19. return;
    → Menghentikan method.
20. System.out.println("Lagu ... berhasil dihapus!");
    → Menampilkan lagu yang dihapus.
21. if (head_3014 == tail_3014) {
    → Jika hanya ada satu lagu.
22. head_3014 = null;
    → Mengosongkan head.
23. tail_3014 = null;
    → Mengosongkan tail.
24. else {
    → Jika lagu lebih dari satu.
25. head_3014 = head_3014.next_3014;
    → Head pindah ke node berikutnya.
26. head_3014.prev_3014 = null;
    → Pointer prev head dikosongkan.

27. public static void tampilMaju_3014() {
    → Method tampil playlist dari depan.
28. Lagu_2511533014 bantu_3014 = head_3014;
    → Pointer bantu mulai dari head.
29. while (bantu_3014 != null) {
    → Perulangan selama node masih ada.
30. System.out.println("Judul : " + ...);
    → Menampilkan judul lagu.
31. System.out.println("Penyanyi : " + ...);
    → Menampilkan penyanyi lagu.
32. bantu_3014 = bantu_3014.next_3014;
    → Pointer maju ke node berikutnya.
33. public static void tampilMundur_3014() {
    → Method tampil playlist dari belakang.
34. Lagu_2511533014 bantu_3014 = tail_3014;
    → Pointer bantu mulai dari tail.
35. while (bantu_3014 != null) {
    → Perulangan selama node masih ada.
36. bantu_3014 = bantu_3014.prev_3014;
    → Pointer bergerak ke node sebelumnya.
37. public static void cariLagu_3014(String judulCari_3014) {
    → Method mencari lagu berdasarkan judul.
38. boolean ditemukan_3014 = false;
    → Penanda apakah lagu ditemukan.
39. if (bantu_3014.getJudul_3014().equalsIgnoreCase(...)) {
    → Mengecek judul lagu tanpa membedakan huruf besar kecil.
40. ditemukan_3014 = true;
    → Menandai lagu ditemukan.
41. break;
    → Menghentikan pencarian.
42. if (ditemukan_3014 == false) {
    → Jika lagu tidak ditemukan.

43. System.out.println("Lagu tidak ditemukan!");
→ Menampilkan pesan lagu tidak ditemukan.
44. public static void main(String[] args) {
→ Method utama program.
45. Scanner input_3014 = new Scanner(System.in);
→ Membuat Scanner untuk input.
46. do {
→ Perulangan menu program.
47. System.out.println("1. Tambah Lagu");
→ Menampilkan menu tambah lagu.
48. pilihan_3014 = input_3014.nextInt();
→ Input pilihan user.
49. if (pilihan_3014 == 1) {
→ Jika memilih tambah lagu.
50. tambahLagu_3014(judul_3014, penyanyi_3014);
→ Memanggil method tambah lagu.
51. else if (pilihan_3014 == 2) {
→ Jika memilih hapus lagu.
52. hapusLaguAwal_3014();
→ Memanggil method hapus lagu.
53. else if (pilihan_3014 == 3) {
→ Jika memilih tampil maju.
54. tampilMaju_3014();
→ Menampilkan playlist maju.
55. else if (pilihan_3014 == 4) {
→ Jika memilih tampil mundur.
56. tampilMundur_3014();
→ Menampilkan playlist mundur.
57. else if (pilihan_3014 == 5) {
→ Jika memilih cari lagu.

58. cariLagu_3014(cari_3014);

→ Memanggil method cari lagu.

59. else if (pilihan_3014 == 6) {

→ Jika memilih keluar.

60. System.out.println("Program selesai!");

→ Menampilkan pesan selesai.

61. else {

→ Jika pilihan salah.

62. System.out.println("Pilihan tidak valid!");

→ Menampilkan pesan error pilihan.

63. } while (pilihan_3014 != 6);

→ Perulangan berhenti jika pilih 6.

64. input_3014.close();

→ Menutup Scanner.

65. }

→ Penutup class.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.10 dibawah ini.

```
<terminated> Musik_2511533014 [Java Application] C:\Users\rar

=== Playlist Musik NIM: 2511533014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist Maju
4. Lihat Playlist Mundur
5. Cari Lagu
6. Keluar
Pilihan : 1
Judul :  Rosalía - Mio Cristo Piange Diamanti
Penyanyi : Rosalía
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511533014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist Maju
4. Lihat Playlist Mundur
5. Cari Lagu
6. Keluar
Pilihan : 1
Judul : Risk It All
Penyanyi : Bruno Mars
Lagu berhasil ditambahkan!
```

```
<terminated> Musik_2511533014 [Java Application] C:\Users\ra
=== Playlist Musik NIM: 2511533014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist Maju
4. Lihat Playlist Mundur
5. Cari Lagu
6. Keluar
Pilihan : 1
Judul : Sesi Potret
Penyanyi : Enau
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511533014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist Maju
4. Lihat Playlist Mundur
5. Cari Lagu
6. Keluar
Pilihan : 3
=== Playlist Maju ===
Judul :  Rosalía - Mio Cristo Piange Diamanti
Penyanyi : Rosalía

Judul : Risk It All
Penyanyi : Bruno Mars

Judul : Sesi Potret
Penyanyi : Enau
```

```
<terminated> Musik_2511533014 [Java Application] C:\Users\ra
=== Playlist Musik NIM: 2511533014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist Maju
4. Lihat Playlist Mundur
5. Cari Lagu
6. Keluar
Pilihan : 4
=== Playlist Mundur ===
Judul : Sesi Potret
Penyanyi : Enau

Judul : Risk It All
Penyanyi : Bruno Mars

Judul :  Rosalía - Mio Cristo Piange Diamanti
Penyanyi : Rosalía
```

```
<terminated> Musik_2511533014 [Java Application] C:\Users\rara0\.p2\pool\plugins\org
=== Playlist Musik NIM: 2511533014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist Maju
4. Lihat Playlist Mundur
5. Cari Lagu
6. Keluar
Pilihan : 5
Masukkan judul lagu : Sesi Potret
Lagu ditemukan!
Judul : Sesi Potret
Penyanyi : Enau

=== Playlist Musik NIM: 2511533014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist Maju
4. Lihat Playlist Mundur
5. Cari Lagu
6. Keluar
Pilihan : 2
Lagu  Rosalía - Mio Cristo Piange Diamanti   berhasil dihapus!
```

```
=== Playlist Musik NIM: 2511533014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist Maju
4. Lihat Playlist Mundur
5. Cari Lagu
6. Keluar
Pilihan : 5
Masukkan judul lagu : Rosalía - Mio Cristo Piange Diamanti
Lagu tidak ditemukan!

=== Playlist Musik NIM: 2511533014 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist Maju
4. Lihat Playlist Mundur
5. Cari Lagu
6. Keluar
Pilihan : 6
Program selesai!
```

Gambar 2.10 Output Kode program Class Musik

2.6.3 Analisis

Berdasarkan output program di atas, program Double Linked List playlist musik telah berjalan dengan baik sesuai fungsi yang dibuat. User berhasil menambahkan tiga lagu ke dalam playlist, kemudian playlist dapat ditampilkan secara maju dari head ke tail

dan secara mundur dari tail ke head, yang membuktikan bahwa pointer next dan prev bekerja dengan benar. Fitur pencarian lagu juga berhasil menemukan lagu “Sesi Potret” menggunakan method pencarian. Setelah lagu pertama dihapus, program berhasil memperbarui posisi head sehingga lagu yang dihapus tidak dapat ditemukan lagi saat dicari kembali. Hal ini menunjukkan bahwa operasi tambah, hapus, tampil, dan cari pada Double Linked List sudah berfungsi dengan benar dan sesuai konsep struktur data DLL.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Double Linked List merupakan salah satu struktur data linear yang memiliki kemampuan traversal dua arah menggunakan pointer next dan prev. Dengan adanya dua pointer tersebut, proses pengolahan data menjadi lebih fleksibel dibandingkan Single Linked List karena data dapat diakses dari depan maupun belakang. Pada praktikum ini, implementasi Double Linked List menggunakan bahasa Java berhasil dilakukan melalui beberapa operasi dasar seperti menambah data, menghapus data, menampilkan data secara maju dan mundur, serta mencari data berdasarkan judul lagu.

Selain itu, praktikum ini juga membantu memahami cara kerja hubungan antar node dalam struktur data dinamis. Penggunaan class node, constructor, getter setter, dan pointer antar node menunjukkan bagaimana data saling terhubung dalam memori. Dengan memahami Double Linked List, mahasiswa dapat meningkatkan pemahaman mengenai struktur data dan penerapannya dalam pembuatan program yang lebih kompleks di masa mendatang.

DAFTAR PUSTAKA

[1] GeeksforGeeks, “Doubly Linked List Data Structure,” 2024. [Online]. Available: https://www.geeksforgeeks.org/doubly-linked-list/. [Accessed: 14-May-2026].

[2] Oracle, “Java Documentation,” 2024. [Online]. Available: https://docs.oracle.com/javase/tutorial/. [Accessed: 14-May-2026].

[3] Programiz, “Doubly Linked List,” 2024. [Online]. Available: https://www.programiz.com/dsa/doubly-linked-list. [Accessed: 14-May-2026].

[4] Tutorialspoint, “Data Structure - Doubly Linked List,” 2024. [Online]. Available: https://www.tutorialspoint.com/data_structures_algorithms/doubly_linked_list_algorithm.htm. [Accessed: 14-May-2026].